David Guillermo Flores.

PROPUESTA DE ARQUITECTURA DE INTEGRACIÓN BANCARIA (AWS)

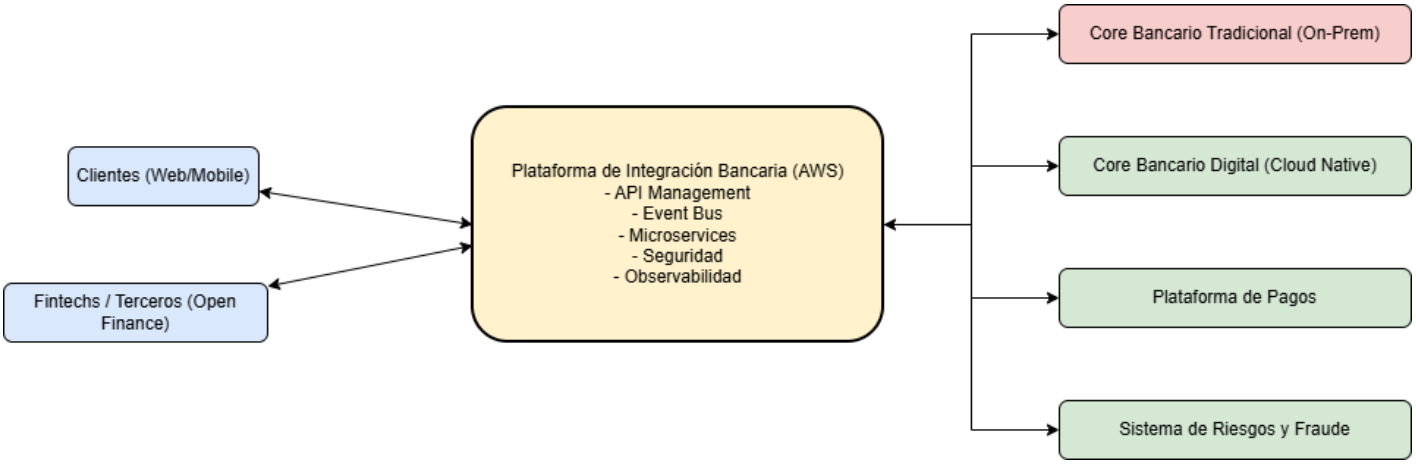
1. Resumen Ejecutivo

Este documento presenta una propuesta de arquitectura de integración para la modernización de un banco tradicional, utilizando servicios administrados de AWS y una arquitectura basada en microservicios y eventos.

La solución permite:

- Coexistencia de core legacy y core digital.
- Integración Open Finance.
- Procesamiento de pagos con validación de riesgo y fraude.
- Alta disponibilidad Multi-AZ.
- Arquitectura desacoplada basada en eventos.
- Cumplimiento normativo y trazabilidad completa.

2. Arquitectura de Alto Nivel (C4-Contexto)



Componentes principales:

- Clientes Web/Mobile
- Fintechs / Open Banking
- Plataforma de Integración Bancaria (AWS)
- Core Legacy (On-Prem)
- Core Digital (Cloud Native)
- Plataforma de Pagos
- Sistema de Riesgos y Fraude

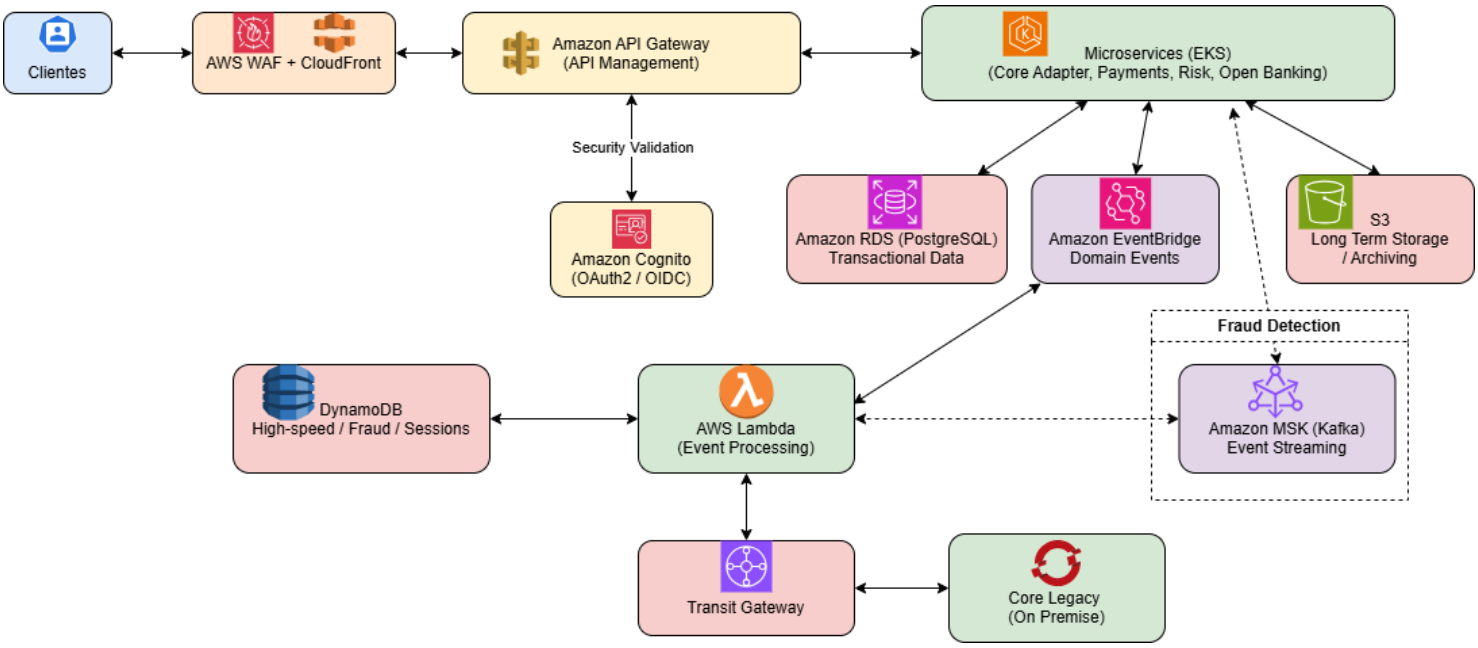
Rol de la Plataforma AWS:

- API Management
- Event Bus
- Microservicios
- Seguridad
- Observabilidad

Justificación:

Se implementa un modelo híbrido donde AWS actúa como capa de integración desacoplada entre sistemas internos y externos.

3. Arquitectura de Contenedores (C4-Containers)



3.1 Capa de Entrada

- CloudFront + WAF, esto para Protección perimetral
- API Gateway para Exposición de APIs
- Amazon Cognito para Autenticación OAuth2/OIDC
- ALB para Balanceo Multi-AZ

Patrones aplicados:

- API Gateway Pattern
- Edge Security Pattern

3.2 Capa de Aplicación

- Amazon EKS (Multi-AZ)
- Microservicios:
    - Payment Orchestrator
    - Risk Service
    - Core Adapter
    - Event Publisher
    - Fraud Service
    - Audit Service

Patrones aplicados:

- Microservices Architecture
- Saga Pattern (Orchestrated)
- Adapter Pattern

3.3 Capa de Datos

- Amazon RDS (CQRS Pattern)
    - Write Instance
    - Read Replica
- DynamoDB (High-speed data / sesiones / fraude)

- S3 (Archiving)

Patrones aplicados:

- CQRS
- Transactional Outbox
- Data Segregation

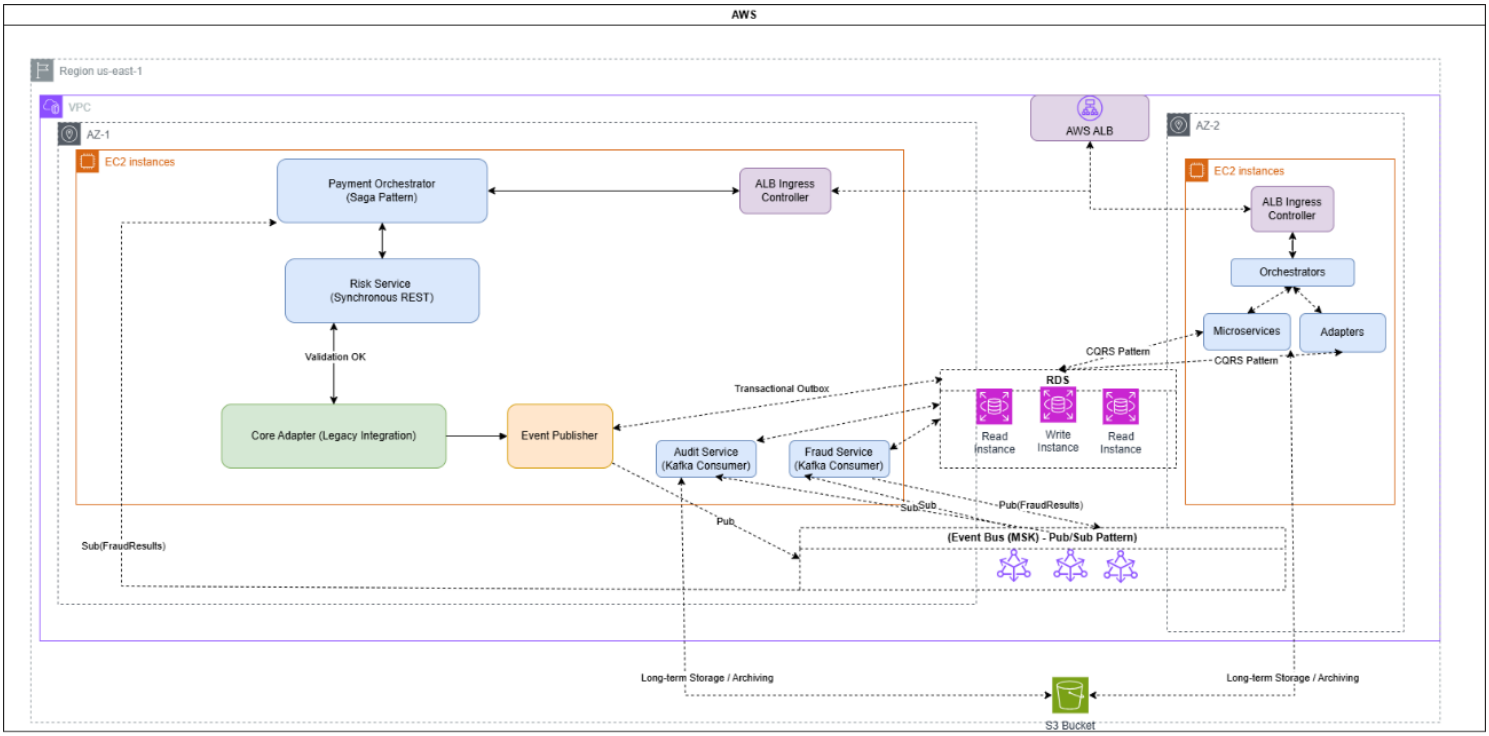
3.4 Capa de Eventos

- Amazon MSK (Kafka)
- EventBridge (Domain Events)

Patrones aplicados:

- Event-Driven Architecture
- Publish / Subscribe
- Event Notification
- Competing Consumers

4. Arquitectura de Componentes (C4-Components)



Flujo de Pago

1. Cliente invoca API.
2. ALB enruta hacia Ingress Controller.
3. Payment Orchestrator inicia Saga.
4. Risk Service valida transacción (REST síncrono).
5. Si validación OK → Core Adapter ejecuta transacción.
6. Se persiste en RDS.
7. Transactional Outbox registra evento.
8. Event Publisher publica en MSK.
9. Fraud Service consume evento.
10. Fraud Service valida y publica en tópico FraudResults.

11. Payment Orchestrator puede consumir del topico FraudResults de manera asíncrona.
12. Audit Service registra evento para cumplimiento.

5. Patrones de Arquitectura Utilizados

Componente	Patrón
MSK	Publish/Subscribe
Payment	Saga Pattern
Core Adapter	Adapter Pattern
RDS	CQRS
Event Publisher	Transactional Outbox
Fraud	Competing Consumers
API Gateway	API Gateway Pattern
Integración Legacy	Strangler Pattern

6. Seguridad y Cumplimiento

6.1 Seguridad de Red

- VPC segmentada
- Subredes privadas para EKS
- ALB Multi-AZ
- WAF
- Transit Gateway para integración on-premise

6.2 Seguridad de Identidad

- Cognito OAuth2/OIDC
- IAM Roles por microservicio
- Principio de mínimo privilegio

6.3 Protección de Datos

- Encriptación en tránsito (TLS 1.2+)
- Encriptación en reposo (KMS), se podría integrar con lambdas y centralizar cifrado AES.
- Auditoría inmutable
- Retención en S3

Cumple con:

- Ley de Protección de Datos
- Regulación financiera
- Trazabilidad obligatoria

7. Alta Disponibilidad y DR

Alta Disponibilidad

- EKS Multi-AZ
- ALB Multi-AZ
- RDS Multi-AZ
- MSK Multi-AZ

SLA estimado ≥ 99.95%

Recuperación ante Desastres

- Backup automático RDS
- Replicación cross-region (opcional, además hay costos por uso de región)
- RPO < 15 min
- RTO < 1 hora

8. Estrategia Multicore

Para minimizar el riesgo operativo:

- Core Legacy encapsulado vía Adapter
- Nuevo Core Digital integrado vía APIs
- Migración gradual (Strangler Pattern)

9. Gobierno de APIs y Microservicios

- **Versionado obligatorio**: todas las APIs deben implementar versionado explícito (por ejemplo: /v1, /v2) para garantizar compatibilidad hacia atrás (backward compatibility) y facilitar la evolución controlada de los servicios.
- **OpenAPI Specification**: documentación estandarizada de las APIs utilizando OpenAPI, permitiendo generación automática de documentación, validación de contratos y generación de clientes.
- **Contract-First**: diseño de APIs basado en contratos previamente definidos (OpenAPI), asegurando alineación entre equipos de desarrollo y evitando inconsistencias entre consumidor y proveedor del servicio.
- **CI/CD automatizado**: pipelines de integración y despliegue continuo que incluyen validación de contratos, pruebas automatizadas, análisis de calidad de código y despliegues controlados en los distintos ambientes.
- **Observabilidad obligatoria**: todos los microservicios deben implementar métricas, logs y trazabilidad distribuida para garantizar monitoreo continuo, diagnóstico de fallos y análisis de rendimiento.
- **SLIs / SLOs definidos**: definición de Service Level Indicators (SLIs) y Service Level Objectives (SLOs) para medir disponibilidad, latencia, tasa de errores y otros indicadores clave del servicio.

10. Plan de Migración Gradual (Strangler Pattern)

Fase 1: Implementación API Gateway + Adapter
Fase 2: Event Enablement (EventBridge, MSK)
Fase 3: Migración de pagos
Fase 4: Migración de cuentas
Fase 5: Desacoplamiento total

11. Observabilidad

- **Amazon CloudWatch**: monitoreo centralizado de métricas, logs y eventos de la infraestructura y servicios desplegados en AWS.
- **AWS X-Ray**: trazabilidad distribuida (distributed tracing) para analizar el flujo de las solicitudes entre microservicios y detectar cuellos de botella o fallos en la arquitectura.
- **Métricas por dominio**: definición de métricas específicas por dominio de negocio (por ejemplo: pagos procesados, validaciones de riesgo, transacciones fallidas), permitiendo monitoreo funcional además del monitoreo técnico.

- **Alertas proactivas**: configuración de alertas basadas en umbrales (threshold-based monitoring) para eventos clasificados como warning y critical, permitiendo detectar anomalías y actuar de forma anticipada antes de que impacten en la disponibilidad o rendimiento del servicio.
- **Logs centralizados**: implementación de centralización de logs mediante ELK Stack (Elasticsearch, Logstash y Kibana). Los logs de las aplicaciones se recolectan mediante contenedores sidecar con Filebeat en los pods, encargados de enviar la información hacia la plataforma de análisis y observabilidad.

12. Beneficios de la Arquitectura

- **Desacoplamiento total**: la arquitectura basada en microservicios y eventos permite que los componentes del sistema operen de forma independiente, facilitando la evolución del sistema sin afectar otros servicios.
- **Escalabilidad horizontal**: los servicios pueden escalar de manera independiente según la demanda, permitiendo manejar picos de carga mediante autoescalado en la infraestructura cloud.
- **Procesamiento en tiempo real**: la arquitectura orientada a eventos permite procesar información y reaccionar a eventos del sistema en tiempo real, mejorando la capacidad de análisis y respuesta.
- **Integración híbrida segura**: soporte para integración entre entornos on-premise y cloud mediante mecanismos seguros de conectividad y control de acceso.
- **Cumplimiento normativo**: la arquitectura incorpora controles de seguridad, trazabilidad y monitoreo que facilitan el cumplimiento de regulaciones y auditorías.
- **Alta disponibilidad y resiliencia Multi-AZ**: despliegue distribuido en múltiples zonas de disponibilidad para garantizar continuidad del servicio ante fallos de infraestructura.